

Day 8 — SQL Fundamentals

SELECT, Filtering, Sorting & NULL Handling — Notes for Recap

All examples below use a sample table called `online_retail`, matching the same retail dataset structure from Day 7: `invoice_no`, `stock_code`, `description`, `quantity`, `invoice_date`, `unit_price`, `customer_id`, `country`.

1. SELECT

`SELECT` is how you choose which columns to retrieve from a table. It is the starting point of almost every SQL query.

```
-- Select specific columns
SELECT invoice_no, description, quantity
FROM online_retail;

-- Select all columns (use sparingly - explicit columns are better practice)
SELECT * FROM online_retail;
```

Key takeaway: Always prefer naming specific columns over `SELECT *` in real pipelines - it's faster, clearer, and won't silently break if the table's columns change later.

2. WHERE

`WHERE` filters rows based on a condition - only rows where the condition is true are returned.

```
-- Single condition
SELECT * FROM online_retail
WHERE country = 'United Kingdom';

-- Multiple conditions
SELECT * FROM online_retail
WHERE country = 'United Kingdom' AND quantity > 10;

-- Range condition
SELECT * FROM online_retail
WHERE unit_price BETWEEN 1.0 AND 5.0;
```

Key takeaway: `WHERE` is the SQL equivalent of pandas row filtering (`df[df['col'] > x]`) that you practiced on Day 5 - same logic, different syntax.

3. ORDER BY

`ORDER BY` sorts the result set by one or more columns, ascending by default.

```
-- Sort by unit_price, highest first
SELECT description, unit_price
FROM online_retail
ORDER BY unit_price DESC;

-- Sort by multiple columns
SELECT country, customer_id, quantity
FROM online_retail
ORDER BY country ASC, quantity DESC;
```

Key takeaway: ASC (default) sorts low-to-high or A-to-Z; DESC reverses it. When sorting by multiple columns, SQL sorts by the first column, then breaks ties using the next one.

4. LIMIT

LIMIT restricts how many rows are returned - useful for previewing data or answering "top N" style questions.

```
-- Preview the first 10 rows
SELECT * FROM online_retail
LIMIT 10;

-- Top 5 highest-priced items
SELECT description, unit_price
FROM online_retail
ORDER BY unit_price DESC
LIMIT 5;
```

Key takeaway: ORDER BY + LIMIT together is the standard pattern for "top N" or "bottom N" questions - one of the most common request types in real analytics work.

5. DISTINCT

DISTINCT removes duplicate rows from the result, returning only unique values.

```
-- Unique list of countries in the dataset
SELECT DISTINCT country
FROM online_retail;

-- Unique combinations of two columns
SELECT DISTINCT country, customer_id
FROM online_retail;
```

Key takeaway: This is the SQL equivalent of converting a pandas column to a set () (Day 6) - same de-duplication idea, applied directly in the query instead of after loading the data.

6. CASE

CASE lets you write conditional logic directly inside a SQL query - the SQL equivalent of an if/elif/else statement (Day 1), producing a new column based on conditions.

```
SELECT
    description,
    quantity,
    CASE
        WHEN quantity >= 50 THEN 'Bulk Order'
        WHEN quantity >= 10 THEN 'Medium Order'
        ELSE 'Small Order'
    END AS order_size
FROM online_retail;
```

Key takeaway: CASE is evaluated top to bottom, just like Python's if/elif/else - the first matching WHEN wins, and ELSE is the fallback if nothing matches.

7. NULL Handling

NULL represents a missing or unknown value - it is not the same as zero or an empty string, and it cannot be compared using =.

```
-- WRONG - this will never match NULL rows, even if customer_id is missing
SELECT * FROM online_retail WHERE customer_id = NULL;

-- CORRECT - use IS NULL / IS NOT NULL
SELECT * FROM online_retail WHERE customer_id IS NULL;
SELECT * FROM online_retail WHERE customer_id IS NOT NULL;

-- Replacing NULLs with a default value
SELECT
    invoice_no,
    COALESCE(customer_id, 0) AS customer_id_filled
FROM online_retail;
```

Key takeaway: This connects directly to what you saw in your Day 7 project - customer_id had ~135,000 missing values. In SQL, you must always use IS NULL / IS NOT NULL to check for missing values, never = or !=.

Day 8 Summary — SQL Fundamentals

Concept	One-Line Recap
SELECT	Choose which columns to retrieve
WHERE	Filter rows based on a condition
ORDER BY	Sort results ascending or descending
LIMIT	Restrict the number of rows returned

DISTINCT	Remove duplicate rows from the result
CASE	Conditional logic inside a query - SQL's if/elif/else
NULL Handling	Use IS NULL / IS NOT NULL - never = or != for missing values

Overall takeaway: These seven building blocks form the base of almost every SQL query you'll write. Notice how closely they map to what you already know from pandas - *WHERE* is filtering, *ORDER BY* is *sort_values*, *DISTINCT* is *set()* - SQL is often the same logic in a declarative form.